Практична робота № 6

Тема: Комунікація в реальному часі засобами SSE та WebSockets

---

Мета роботи

- Опанувати технологію Server-Sent Events (SSE) для організації однонапрямленої потокової доставки подій та телеметрії від сервера до клієнта через HTTP-інтерфейс `http.Flusher`.
- Засвоїти специфікацію повнодуплексного протоколу WebSocket (RFC 6455) та механізм переведення протоколів (HTTP Upgrade) за допомогою бібліотеки `gorilla/websocket`.
- Навчитися реалізовувати масштабований архітектурний патерн Hub для централізованої реєстрації, дереєстрації клієнтів та широкомовної розсилки повідомлень (Broadcasting) через канали Go.
- Опанувати розділення логіки обробки з'єднань на незалежні горутини читання (`readPump`) та потокобезпечного запису (`writePump`).
- Реалізувати тематичні кімнати комунікації (Rooms) та механізм контролю життєздатності з'єднань (Heartbeat Ping/Pong).

Технічне забезпечення

- Операційна система: Windows, macOS або Linux.
- Середовище розробки: Visual Studio Code (з офіційним розширенням Go) або JetBrains GoLand.
- Компілятор: Go SDK версії 1.22 або новішої.
- Термінал / CLI: bash, zsh або PowerShell.
- Система контролю версій: Git.
- Інструменти тестування: утиліти `curl` (з прапорцем `-N` для потокових відповідей), CLI-клієнт WebSocket (`wscat` або `websocat`) або сучасний веббраузер.

Короткі теоретичні відомості

1. Однонапрямлений потік подій Server-Sent Events (SSE)

Server-Sent Events (SSE) — це стандарт передачі текстових повідомлень від сервера до клієнта через єдине довготривале HTTP-з'єднання. Клієнт відкриває стандартний HTTP GET-запит, а сервер утримує з'єднання відкритим, надсилаючи нові фрагменти даних у міру їх появи.

Ключові технічні вимоги до SSE:

- HTTP-заголовки:
  - `Content-Type: text/event-stream` — інформує клієнта про потоковий формат;
  - `Cache-Control: no-cache` — забороняє кешування проміжними проксі-серверами;
  - `Connection: keep-alive` — утримує TCP-сесію відкритою.
- Формат повідомлень: текстовий блок, що починається з префікса `data:` та завершується подвійним переводом рядка `\n\n`.
- Інтерфейс `http.Flusher`: стандартний механізм Go для примусового виштовхування буферизованих байтів безпосередньо в мережевий сокет без закриття відповіді.

```
1 flusher, ok := w.(http.Flusher) ①
2 if !ok {
3     http.Error(w, "Streaming unsupported", http.StatusInternalServerError)
4     return
5 }
6
7 w.Header().Set("Content-Type", "text/event-stream") ②
8 w.Header().Set("Cache-Control", "no-cache")
9 w.Header().Set("Connection", "keep-alive")
10
11 // Надсилання події
12 fmt.Fprintf(w, "data: %s\n\n", `{"status":"active"}`) ③
13 flusher.Flush() ④
```

① Перевірка підтримки примусового виштовхування буфера через приведення до інтерфейсу `http.Flusher`.

② Конфігурація службових заголовків довготривалого потокового з'єднання.

③ Формування повідомлення згідно зі стандартом SSE з подвійним переносом рядка.

④ Негайне відправлення даних клієнту без очікування заповнення внутрішнього буфера сервера.

2. Протокол WebSocket та процедура оновлення з'єднання (Upgrade)

На відміну від SSE, **WebSocket** є повнодуплексним протоколом, що забезпечує двосторонній обмін фреймами між клієнтом і сервером з мінімальними накладними витратами (розмір заголовка фрейму становить лише 2–14 байтів).

Встановлення з'єднання починається зі звичайного HTTP-запиту клієнта з заголовками:

- `Upgrade: websocket`
- `Connection: Upgrade`
- `Sec-WebSocket-Key: <base64_ключ>`

У Go для перетворення HTTP-з'єднання на повноцінний WebSocket застосовується об'єкт `websocket.Upgrader` з бібліотеки `github.com/gorilla/websocket`:

```
var upgrader = websocket.Upgrader{
    CheckOrigin: func(r *http.Request) bool { ①
        return true // Дозволяє підключення з будь-яких джерел (CORS)
    },
}

func wsHandler(w http.ResponseWriter, r *http.Request) {
    conn, err := upgrader.Upgrade(w, r, nil) ②
    if err != nil {
        log.Printf("Upgrade failed: %v", err)
        return
    }
    defer conn.Close()
}
```

① Налаштування політики безпеки джерел (дозволяє підключення вебклієнтам).

② Виконання процедури рукостискання (HTTP 101 Switching Protocols) та перехід на протокол WebSocket.

---

3. Архітектурний патерн Hub та потокобезпека з'єднань

Об'єкт `websocket.Conn` ***не є потокобезпечним*** для одночасного виклику операцій запису з декількох горутин. Для безпечного масштабування використовується патерн **Hub**:

1. **Горутина `readPump`:** безперервно вичитує повідомлення з сокета (`conn.ReadMessage()`) та передає їх у канал диспетчера `hub.broadcast`.
2. **Горутина `writePump`:** вичитує повідомлення з буферизованого каналу клієнта `client.send` і

послідовно записує їх у сокет (`conn.WriteMessage()`).

3. **Диспетчер `Hub`:** працює в окремій горутині, координуючи додавання клієнтів, їх видалення та розсилку повідомлень за допомогою оператора `select`.

---

4. Контроль з'єднань Heartbeat (Ping/Pong)

Для виявлення "завислих" клієнтів (наприклад, раптове вимкнення пристрою або втрата Wi-Fi без надсилання FIN-пакета) використовується періодичне надсилання Ping-фреймів. Якщо клієнт не надсилає відповідний Pong-фрейм у межах встановленого дедлайну (`SetReadDeadline`), з'єднання примусово закривається, звільняючи пам'ять і дескриптори сокетів на сервері.

Постановка задачі

У диспетчерських системах оперативного керування (наприклад, моніторинг серверної інфраструктури або пульт чергового інженера) виникає потреба поєднання двох різних режимів передачі даних:

1. **Однонапрямлена трансляція телеметрії (SSE):** чергова вебпанель повинна безперервно отримувати системні метрики сервера (навантаження, кількість активних горутин, мітки часу) з мінімальними витратами ресурсів;
2. **Інтерактивна координація операторів (WebSockets):** інженери повинні мати двосторонній канал оперативного зв'язку та оповіщення з можливістю підключення до тематичних кімнат (наприклад, кімната термінових аварій або загальний чат) та миттєвим поширенням повідомлень усім активним учасникам.

Необхідно розробити **сервіс моніторингу та оперативного зв'язку реального часу (Real-Time Operations Gateway)**, що поєднує в собі SSE-стрімінг метрик, повнодуплексний WebSocket-сервер на базі патерну Hub із підтримкою тематичних кімнат, механізм контролю з'єднань Heartbeat та тестову клієнтську вебсторінку для перевірки.

Завдання

Практична робота складається з трьох послідовних рівнів. Кожен наступний рівень базується на попередньому та розширює функціональність розроблюваного рішення.

Рівень 1. Базовий: Однонапрямлений потік телеметрії через Server-Sent Events (SSE)

1. **Ініціалізація проєкту:**

- Створити окрему робочу директорію для практичної роботи.
- Ініціалізувати Go-модуль за допомогою `go mod init <назва_модуля>`.
- Створити точку входу `main.go`.

2. **Реалізація SSE-обробника потокових метрик:**
   - Зареєструвати HTTP-маршрут `GET /api/v1/metrics/stream`.
   - Перевірити підтримку потокового виведення через приведення типу до інтерфейсу `http.Flusher`.
   - Встановити обов'язкові службові заголовки: `text/event-stream`, `no-cache`, `keep-alive`.
3. **Організація циклу трансляції подій:**
   - Запустити періодичний таймер на базі `time.NewTicker(1 * time.Second)`.
   - Щосекунди зчитувати поточний стан системи за допомогою стандартного пакета `runtime`:
     - поточний час сервера (`time.Now().Format(time.RFC3339)`);
     - кількість активних горутин у процесі (`runtime.NumGoroutine()`);
     - кількість виділеної оперативної пам'яті (`runtime.MemStats`).
   - Серіалізувати метрики у формат JSON та відправити у потік згідно зі специфікацією SSE:

     ```
     data: {"time": "...", "goroutines": 12, "alloc_mb": 4.2}
     ```

   - Викликати `flusher.Flush()` після кожного запису для миттєвої доставки даних клієнту.
4. **Контроль життєвого циклу клієнтського запиту:**
   - Організувати очікування розриву зв'язку клієнтом за допомогою каналу `<-r.Context().Done()`.
   - При закритті вкладки або завершенні запиту клієнтом коректно зупиняти таймер (`ticker.Stop()`) та завершувати обробник без витоку горутин у пам'яті.
5. **Тестування:**
   - Перевірити неперервний потік подій у терміналі за допомогою утиліти `curl` без буферизації:

     ```
     1 curl -N http://localhost:8080/api/v1/metrics/stream
     ```

   - Переконатися, що нові рядки з'являються щосекунди без переривання сесії.

Рівень 2. Ключовий: Повнодуплексний WebSocket-сервер та патерн Hub (Broadcast)

1. **Збереження функціональності Рівня 1.**
2. **Встановлення та налаштування бібліотеки Gorilla WebSocket:**
   - Встановити залежність:

     ```
     go get github.com/gorilla/websocket
     ```

   - Ініціалізувати об'єкт `websocket.Upgrader` з налаштуванням `CheckOrigin` для дозволу з'єднань із будь-яких доменів.
3. **Реалізація архітектурного патерну Hub:**
   - Описати структуру `Hub`:
     - мапа зареєстрованих клієнтів `clients map[*Client]bool`;
     - канал для широкомовних повідомлень `broadcast chan []byte`;
     - канал для реєстрації нових клієнтів `register chan *Client`;
     - канал для видалення клієнтів `unregister chan *Client`.
   - Реалізувати метод `Run()` для диспетчера Hub, який у нескінченному циклі через оператор `select` обробляє:
     - додавання нового клієнта до мапи;
     - видалення клієнта із закриттям його персонального каналу відправки;
     - отримання повідомлення з каналу `broadcast` та його ретрансляцію в канали `send` усіх підключених клієнтів.
   - Запустити диспетчер Hub в окремій горутині у функції `main()`.
4. **Проєктування клієнтського обробника (Client Pumping):**
   - Описати структуру `Client`: посилання на `Hub`, з'єднання `*websocket.Conn` та буферизований канал відправки `send chan []byte` (місткістю 256 повідомлень).
   - Реалізувати метод `readPump()`:
     - у циклі вичитувати повідомлення з сокета за допомогою `conn.ReadMessage()`;
     - при отриманні повідомлення передавати його в `hub.broadcast`;
     - при виникненні помилки читання або закритті сокета ініціювати дереєстрацію клієнта через `hub.unregister`.
   - Реалізувати метод `writePump()`:
     - вичитувати повідомлення з каналу `client.send`;
     - записувати їх у WebSocket через `conn.WriteMessage(websocket.TextMessage, message)`.
5. **Створення ендпоінту підключення:**
   - Зареєструвати маршрут `GET /api/v1/ws`.

- Виконувати апгрейд з'єднання через `upgrader.Upgrade`.
- Створювати екземпляр `Client`, реєструвати його в Hub, запускати `writePump()` в окремій горутині та блокуюче викликати `readPump()`.

6. **Тестування:**
   - Відкрити два окремі термінали та підключитися через утиліту `wscat` (або `websocat`):

     ```
     1 wscat -c ws://localhost:8080/api/v1/ws
     ```

   - Надіслати тестове повідомлення з першого термінала та переконатися, що воно миттєво з'являється у другому терміналі.

Рівень 3. Просунутий: Тематичні кімнати (Rooms), Heartbeat Ping/Pong та вебклієнт

1. **Збереження функціональності попередніх рівнів.**
2. **Організація тематичних кімнат комунікації (Room Management):**
   - Додати до структури `Client` текстове поле `room string`.
   - При підключенні витягувати назву кімнати з query-параметра запиту (наприклад, `/api/v1/ws?room=alerts` або `/api/v1/ws?room=general`). Якщо параметр відсутній — призначати кімнату за замовчуванням `general`.
   - Модернізувати структуру повідомлення, додавши інформацію про цільову кімнату та відправника.
   - У диспетчері Hub ретранслювати отримане повідомлення виключно тим клієнтам, які належать до тієї ж кімнати, що й відправник.
3. **Контроль життєздатності з'єднань (Heartbeat Ping/Pong):**
   - Налаштувати періодичне надсилання службових Ping-фреймів сервером:
     - у горутині `writePump` створити тикер з інтервалом 30–50 секунд;
     - надсилати фрейм `conn.WriteMessage(websocket.PingMessage, nil)`.
   - У горутині `readPump` налаштувати дедлайн читання (`conn.SetReadDeadline`) та обробник відповідей Pong (`conn.SetPongHandler`):
     - при отриманні кожного Pong-відгуку від клієнта продовжувати термін дедлайну читання;
     - якщо клієнт не відповів протягом таймауту — виходити з циклу читання, примусово закриваючи "завислий" сокет.
4. **Створення демонстраційної вебсторінки (Web Client):**
   - Створити окрему статичну HTML-вебсторінку (наприклад, `web/index.html`), яка містить два інтерактивні блоки:

- **Панель телеметрії (SSE):** підключається через стандартний браузерний API `new EventSource('/api/v1/metrics/stream')` та в реальному часі оновлює показники пам'яті й горутин;
- **Пульт оперативного зв'язку (WebSocket):** поле вибору кімнати (`room`), підключення через `new WebSocket(⋯)`, поле введення повідомлень та вікно логу чату.

◦ Налаштувати віддачу статичних файлів у сервері Go (`http.FileServer` або роут `r.Static`).

◦ Відкрити вебсторінку у двох вкладках браузера, підключитися до однакових і різних кімнат та продемонструвати викладачеві роздільну розсилку подій.

Приклад очікуваного виведення програми

Консоль сервера моніторингу та зв'язку:

```
=== Сервіс реального часу (SSE & WebSockets Gateway) ===
[СЕРВЕР] Диспетчер WebSocket Hub успішно запущено
[СЕРВЕР] HTTP-слухач відкрито на порту :8080
[SSE] Нове підключення клієнта телеметрії: 127.0.0.1:53210
[WS] Новий клієнт підключився до кімнати [alerts]: 127.0.0.1:53225
[WS] Новий клієнт підключився до кімнати [alerts]: 127.0.0.1:53240
[WS] Новий клієнт підключився до кімнати [general]: 127.0.0.1:53255
[HUB BROADCAST] [Кімната: alerts] Отримано повідомлення: "Увага! Перегрів вузла node-
03"
[HUB BROADCAST] Сповіщено клієнтів у кімнаті [alerts]: 2
[HEARTBEAT] Ping надіслано клієнту 127.0.0.1:53225 -> Pong отримано успішно
[SSE] Клієнт 127.0.0.1:53210 розірвав з'єднання. Очищення ресурсів виконано.
```

Консоль термінала з клієнтом SSE (`curl -N http://localhost:8080/api/v1/metrics/stream`):

```
data: {"time":"2026-09-08T15:00:01Z","goroutines":8,"alloc_kb":2450}

data: {"time":"2026-09-08T15:00:02Z","goroutines":8,"alloc_kb":2458}

data: {"time":"2026-09-08T15:00:03Z","goroutines":9,"alloc_kb":2465}
```

Консоль терміналів WebSocket (`wscat -c "ws://localhost:8080/api/v1/ws?room=alerts"`):

Клієнт 1:

```
Connected (press CTRL+C to quit)
> Увага! Перегрів вузла node-03
< [alerts] Увага! Перегрів вузла node-03
```

Клієнт 2 (підключений до тієї ж кімнати `alerts`):

```
Connected (press CTRL+C to quit)
< [alerts] Увага! Перегрів вузла node-03
```

Контрольні питання

1. Чим принципово відрізняється архітектура Server-Sent Events (SSE) від повнодуплексного протоколу WebSocket?
2. Яку роль відіграє HTTP-інтерфейс `http.Flusher` при реалізації потокової передачі даних у Go?
3. Що станеться з горутиною обробника SSE, якщо не відстежувати скасування контексту `r.Context().Done()` після відключення клієнта?
4. Опишіть процедуру рукостискання (Upgrade Handshake) при переході з протоколу HTTP на WebSocket.
5. Чому об’єкт `*websocket.Conn` не є безпечним для одночасного запису з кількох горутин і як патерн Hub вирішує цю проблему?
6. Навіщо обробку кожного WebSocket-з’єднання розділяти на дві незалежні горутини (`readPump` та `writePump`)?
7. Як працює механізм Heartbeat на базі Ping/Pong фреймів і чому він необхідний для виявлення розривів зв’язку?
8. Які переваги та обмеження має браузерний API `EventSource` порівняно з нативним об’єктом `WebSocket`?

Критерії оцінювання

Оцінювання здійснюється за шкалою повноти реалізації рівнів практичної роботи:

- **«Задовільно» (Рівень 1):** реалізовано базовий прототип, створено структуру проєкту,

програма успішно компілюється та виконує базові сценарії.

- **«Добре» (Рівень 2):** виконано базовий рівень та реалізовано ключову функціональність заняття, забезпечено надійність і безпеку виконання коду.
- **«Відмінно» (Рівень 3):** повністю завершено всі три рівні завдання з дотриманням кращих інженерних практик, чистоти архітектури та оптимізації.
- **Усний захист:** обов'язкове пояснення логіки коду та впевнені відповіді на контрольні питання викладача.

Вимоги до звітності

1. Опублікувати вихідний код у Git-репозиторії.
2. Оформити файл `README.md` з описом проєкту, інструкцією запуску та обов'язковими скриншотами роботи програми (включаючи демонстрацію відсутності помилок).
3. Продемонструвати роботу програми викладачеві та захистити результати.